In [ ]:
```
!python --version
!pip freeze
# szukamy wersji numpy -> dokumentacja: https://numpy.org/doc/
```

1. TWORZENIE I PRZEKSZTAŁCANIE TABLICY

In [2]:
```
import numpy as np

array = np.array([1,2,3,4,5,6,7,8,9,10,11,12])

print(f"Array")
print(array)
print(f"Shape of array: {np.shape(array)}")

# ? (W jaki inny sposób możemy sprawdzić rozmiar tablicy? - Variables)
# ? (Dlaczego rozmiar tablicy jest zapisany jako (12,) a nie jako (12)? - shape to zawsze krotka)


# ? (Jaka metody służy do zmiany rozmiaru tablicy?)
array2x6 = array.reshape(2,6)

print("Array 2x6")
print(array2x6)
print(f"Shape of array: {np.shape(array2x6)}")
```
```
Array
[ 1  2  3  4  5  6  7  8  9 10 11 12]
Shape of array: (12,)
Array 2x6
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]]
Shape of array: (2, 6)
```

In [3]:
```
# ! Zadanie - Korzystając z metody reshape() zmień rozmiar tabeli kolejno na (3,4) i (2,2,3).
array3x4 = array.reshape(3,4)
print("Array 3x4")
print(array3x4)
print(f"Shape of array: {np.shape(array3x4)}")

array2x2x3 = array.reshape(2,2,3)
print("Array 2x2x3")
print(array2x2x3)
print(f"Shape of array: {np.shape(array2x2x3)}")
```
```
Array 3x4
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
Shape of array: (3, 4)
Array 2x2x3
[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
Shape of array: (2, 2, 3)
```

In [4]:
```
# ? ! Zagadka - Jaki kształt będą miały tablice zagadka1 i zagadka2?
# ('-1' - automatyczne uzupełnienie wymiaru)
zagadka1=array.reshape(-1)
print("zagadka1")
print(zagadka1)
print(f"Shape of zagadka1: {np.shape(zagadka1)}")

zagadka2=array.reshape(6,-1)
print("zagadka2")
print(zagadka2)
print(f"Shape of zagadka2: {np.shape(zagadka2)}")
```
```
zagadka1
[ 1  2  3  4  5  6  7  8  9 10 11 12]
Shape of zagadka1: (12,)
zagadka2
[[ 1  2]
 [ 3  4]
 [ 5  6]
 [ 7  8]
 [ 9 10]
 [11 12]]
Shape of zagadka2: (6, 2)
```

2. SORTOWANIE TABLIC

In [5]:
```
# ? (Czy pamiętacie jaka funkcja służyła do sortowania tablic?)
# numpy.sort - dokumentacja: https://numpy.org/doc/2.0/reference/generated/numpy.sort.html#numpy-sort

# | Argument | Co robi                                              |
# | -------- | ---------------------------------------------------- |
# | `a`      | tablica do posortowania                              |
# | `axis`   | wybiera, czy sortujesz wiersze, kolumny albo całość  |
# | `kind`   | wybiera algorytm sortowania                          |
# | `order`  | sortowanie po polach w tablicy strukturalnej         |
# | `stable` | wymusza stabilne sortowanie                          |

# Ćwiczenia
# 1. Utworzyć tablicę unsorted1D (jednowymiarową) i wyświetlić
import numpy as np
unsorted1D = np.array([3,6,2,4])
print(f"Nieposortowana tablica {unsorted1D}")

# 2. Posortować i wyświetlić
sorted1D = np.sort(unsorted1D)
print(f"Posortowana tablica {sorted1D}")

# 3. Utworzyć 2-wymiarową tablicę o nazwie unsorted2D i wyświetlić
unsorted2D = np.array([[7, 2, 9],
                       [4, 8, 1],
                       [5, 6, 3]])
print(f"Nieposortowana tablica {unsorted2D}")

# 4. Posortuj i wyświetl. Wykonaj sortowanie raz pokrawędzi 0, a drugi raz po krawędzi 1. Czy wynik sortowania się różni?
sorted2D_axis0 = np.sort(unsorted2D,axis=0) # sortowanie w kolumnach
print(f"Tablica posortowana wzdłuż krawędzi 0:\n{sorted2D_axis0}")

sorted2D_axis1 = np.sort(unsorted2D,axis=1) # sortowanie w wierszach
print(f"Tablica posortowana wzdłuż krawędzi 1:\n{sorted2D_axis1}")
```
```
Nieposortowana tablica [3 6 2 4]
Posortowana tablica [2 3 4 6]
Nieposortowana tablica [[7 2 9]
 [4 8 1]
 [5 6 3]]
Tablica posortowana wzdłuż krawędzi 0:
[[4 2 1]
 [5 6 3]
 [7 8 9]]
Tablica posortowana wzdłuż krawędzi 1:
[[2 7 9]
 [1 4 8]
 [3 5 6]]
```

In [6]:
```
# 5.Tworzymy ustrukturyzowaną tablicę z zawodnikami LoLa (League of Legends. Jeżeli grupa pracuje sprawnie może stworzyć własną tablicę)
# opcja 1
players_data = np.array([
("Faker", "T1", 5),
("TheShy", "Weibo Gaming", 1),
("Uzi", "Emerytowany", 0),
("Ruler", "JD Gaming", 1),
("Doinb", "Emerytowany", 1),
("Rekkles", "Los Ratones", 1),
("Caps", "G2 Esports", 0)
], dtype= [
    ('Nick', 'U30'),
    ('Team', 'U30'),
    ('Worlds Title Count', 'i4')
])

# opcja 2
series_data = np.array([
    ("Breaking Bad", "Drama", 5),
    ("Dark", "Sci-Fi", 3),
    ("Friends", "Comedy", 10),
    ("Sherlock", "Crime", 4),
    ("The Office", "Comedy", 9),
    ("Stranger Things", "Sci-Fi", 4),
    ("The Crown", "Drama", 6)
], dtype=[
    ('Title', 'U40'),
    ('Genre', 'U30'),
    ('Seasons', 'i4')
])

# opcja 3
cities_data = np.array([
    ("Poznań", "Polska", 540000),
    ("Berlin", "Niemcy", 3700000),
    ("Praga", "Czechy", 1300000),
    ("Paryż", "Francja", 2100000),
    ("Rzym", "Włochy", 2800000),
    ("Madryt", "Hiszpania", 3200000),
    ("Wiedeń", "Austria", 1900000)
], dtype=[
    ('City', 'U30'),
    ('Country', 'U30'),
    ('Population', 'i4')
])

# dtype - wytłumaczenie
# | Oznaczenie | Znaczenie                    | Przykład            |
# | ---------- | ---------------------------- | ------------------- |
# | `U`        | tekst Unicode                | ``'U30'``           |
# | `S`        | tekst bajtowy                | ``'S10'``           |
# | `i`        | liczba całkowita ze znakiem  | ``'i4'``            |
# | `u`        | liczba całkowita bez znaku   | ``'u4'``            |
# | `f`        | liczba zmiennoprzecinkowa    | ``'f8'``            |
# | `b`        | wartość logiczna / boolean   | ``'?'`` albo `bool` |
# | `c`        | liczba zespolona             | ``'c16'``           |
# | `M`        | data/czas                    | ``'datetime64[D]'`` |
# | `m`        | różnica czasu                | ``'timedelta64[D]'``|
# | `O`        | obiekt Pythona               | `object`            |

# ? (Co by się stało gdybyśmy nie zdefiniowali 'dtype')


# 6. Wyświetl tablicę posortowaną kolejno kolumnami
print("Sortowanie po nicku")
print(np.sort(players_data,order='Nick'))

print("Sortowanie po drużynie")
print(np.sort(players_data,order='Team'))

print("Sortowanie po liczbie tytułów")
print(np.sort(players_data,order='Worlds Title Count'))
```
```
Sortowanie po nicku
[('Caps', 'G2 Esports', 0) ('Doinb', 'Emerytowany', 1) ('Faker', 'T1', 5)
 ('Rekkles', 'Los Ratones', 1) ('Ruler', 'JD Gaming', 1)
 ('TheShy', 'Weibo Gaming', 1) ('Uzi', 'Emerytowany', 0)]
Sortowanie po drużynie
[('Doinb', 'Emerytowany', 1) ('Uzi', 'Emerytowany', 0)
 ('Caps', 'G2 Esports', 0) ('Ruler', 'JD Gaming', 1)
 ('Rekkles', 'Los Ratones', 1) ('Faker', 'T1', 5)
 ('TheShy', 'Weibo Gaming', 1)]
Sortowanie po liczbie tytułów
[('Caps', 'G2 Esports', 0) ('Uzi', 'Emerytowany', 0)
 ('Doinb', 'Emerytowany', 1) ('Rekkles', 'Los Ratones', 1)
 ('Ruler', 'JD Gaming', 1) ('TheShy', 'Weibo Gaming', 1)
 ('Faker', 'T1', 5)]
```

3. LOSOWANIE LICZB

In [7]:
```
# Moduł random z NumPy (mamy dwa różne moduły random)

# | Co chcesz zrobić                   | Lepiej użyć               |
# | ---------------------------------- | ------------------------- |
# | wylosować jedną liczbę             | `random` albo `np.random` |
# | wylosować listę/tablicę wielu liczb | `np.random`              |
# | pracować z tablicami NumPy         | `np.random`               |
# | proste losowanie w zwykłym Pythonie | `random`                 |

# numpy.random.randint - dokumentacja: https://numpy.org/doc/2.0/reference/random/generated/numpy.random.randint.html
# numpy.random.rand - dokumentacja: https://numpy.org/doc/2.0/reference/random/generated/numpy.random.rand.html

from numpy import random

# 1. W pętli 10 razy wyświetlamy liczbę z zakresu od 10 do 20
print("Wylosowanie 10 liczb od 1 do 10")
for i in range(10):
  print(random.randint(10,21))


# 2. w pętli 10 razy wyświetlamy liczbę z zakresu od 0 do 1.
print("Wylosowanie 10 liczb od 0 do 1")
for i in range(10):
  print(random.rand())


# 3. Losowe wartości z list:
# numpy.random.choice - dokumentacja: https://numpy.org/doc/2.0/reference/random/generated/numpy.random.choice.html#numpy-random-choice
# Stwórzmy listę z datami (zagadka dla uczniów czy wiedzą
# z czym związane są te daty (polskie bitwy)?) a następnie wyświetlmy losową
# datę, oraz 3 losowe daty.
daty = [1241, 972, 1605, 1444, 1683]
print(f"Zbiór dat {daty}")
print(f"Losowa data ze zbioru {random.choice(daty)}")
print(f"Losowe 3 losowe daty ze zbioru: {random.choice(daty, 3)}")


# 4. Losowe tasowanie wartość
# numpy.random.shuffle - dokumentacja: https://numpy.org/doc/2.0/reference/random/generated/numpy.random.shuffle.html#numpy-random-shuffle
random.shuffle(daty)
print(f"Daty w losowej kolejności {daty}")
```
```
Wylosowanie 10 liczb od 1 do 10
12
17
13
15
19
11
19
10
18
15
Wylosowanie 10 liczb od 0 do 1
0.2235251616467726
0.35909693479705396
0.20665395896655214
0.5313058725764018
0.006112231683154667 5
0.7380162383657701
0.9279057164124137
0.22616768364691464
0.4700919510152728
0.033993575325712344
Zbiór dat [1241, 972, 1605, 1444, 1683]
Losowa data ze zbioru 1444
Losowe 3 losowe daty ze zbioru: [1683 1444 1605]
Daty w losowej kolejności [972, 1605, 1444, 1241, 1683]
```

4. WYCINKI TABLIC

In [8]:
```
# ! Zadanie
# Utwórzcie tablicę zawierającą liczby od 1 do 9. Następnie należy wyświetlić
# wycinek zawierający:
#   • Elementy od indeksu 2 do indeksu 6 (włącznie)
#   • Elementy do indeksu 4(włącznie)
#   • Elementy od indeksu 5
#   • Elementy od elementu 4 od końca do przedostatniego

# Przypominajka [start:stop:step]
import numpy as np
array1D = np.array([1,2,3,4,5,6,7,8,9])
print(array1D[2:7])
print(array1D[:5])
print(array1D[5:])
print(array1D[-4:-1])
```
```
[3 4 5 6 7]
[1 2 3 4 5]
[6 7 8 9]
[6 7 8]
```

In [9]:
```
# Ćwiczenia
import numpy as np

# 1. Tworzymy tablicę dwuwymiarową zawierającą z tablice z wartościami: 1-5, 6-10 i 11 -15.
array2D =np.array([[1,2,3,4,5],
                   [6,7,8,9,10],
                   [11,12,13,14,15]])

# *Jeżeli chcemy dostać się do pojedynczej wartości podajemy dwa indeksy (tak
# jakbyśmy podawali współrzędne y,x aby określić położenie punktu w układzie współrzędnych).

#           |kolumna 0  | kolumna 1 | kolumna 2 | kolumna 3 |  kolumna 4
# wiersz 0  |    1      |     2     |     3     |     4     |     5
# wiersz 1  |    6      |     7     |     8     |     9     |     10
# wiersz 2  |   11      |    12     |    13     |    14     |     15

print(f"{array2D[2,1]}\n")


# 2. Dla każdego wiersza wygenerujmy wycinek zawierający element o indeksach 2 i 3 w każdym wierszu
# [zakres wierszy, zakres kolumn]
print(f"{array2D[:,2:4]}\n")


# ? Co będzie wynikiem?
print(array2D[1:3,:2])
```
```
12

[[ 3  4]
 [ 8  9]
 [13 14]]

[[ 6  7]
 [11 12]]
```

5. ITEROWANIE PO ELEMENTACH

In [10]:
```
import numpy as np
array2D =np.array([[1,2,3,4,5],
                   [6,7,8,9,10],
                   [11,12,13,14,15]])

for i in array2D:
  for j in i:
    print(j)

print("\n")

# numpy.nditer - dokumentacja: https://numpy.org/doc/2.0/reference/generated/numpy.nditer.html#numpy.nditer
for i in np.nditer(array2D):
  print(i)
```
```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15


1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
```

6. PRZESZUKIWANIE TABLIC

In [11]:
```
# np.where(warunek, wartość_jeśli_prawda, wartość_jeśli_fałsz)
import numpy as np

x = np.array([1, 5, 8, 2, 10])

wynik = np.where(x > 5, "duże", "małe")
print(wynik)

# albo
# wyszukuje po warunku i zwraca indeksy
array2D =np.array( [[1,2,3,4,5],
                    [6,7,8,9,10],
                    [11,12,13,14,6]])
index6 = np.where(array2D == 6)
print(index6)

# Ćwiczenie
# 1. Wyświetlamy indeksy wartości parzystych.
inexes_even = np.where(array2D % 2 == 0)
print(inexes_even) # (wiersze), (kolumny)

# 2. Tworzymy maskę binarną dla wartości parzystych. Oznacza to że wszystkie
# wartości parzyste zastępujemy wartością 1, a nieparzyste wartością 0.
inexes_even = np.where(array2D % 2 == 0,1,0)
print(inexes_even)
```
```
['małe' 'małe' 'duże' 'małe' 'duże']
(array([1, 2]), array([0, 4]))
(array([0, 0, 1, 1, 1, 2, 2, 2]), array([1, 3, 0, 2, 4, 1, 3, 4]))
[[0 1 0 1 0]
 [1 0 1 0 1]
 [0 1 0 1 1]]
```

7. FILTROWANIE TABLIC

In [12]:
```
array2D = np.array([
    [1, 2, 3, 4, 5],
    [6, 7, 8, 9, 10],
    [11, 12, 13, 14, 6]
])
# Chcemy wyciągnąć tylko parzyste

# Krok 1 — tworzymy maskę logiczną
binary_mask = np.where(array2D % 2 == 0, True, False)
print(binary_mask) # kopiujemy wynik

# Krok 2 — filtrowanie tablicy maską
filtered = array2D[binary_mask]
print(filtered)

# ? Dlaczego wynik jest jednowymiarowy?
# - podczas filtrowania NumPy po prostu zbiera wszystkie pasujące elementy po kolei i wrzuca je do jednej tablicy
```
```
[[False  True False  True False]
 [ True False  True False  True]
 [False  True False  True  True]]
[ 2  4  6  8 10 12 14  6]
```

8. PO CO MI TO? - czyli jak NumPy oszczędza nam czas w AI

In [13]:
```
# Ćwiczenie
# 1.Zaczynamy od napisania dekoratora - funkcji która będzie liczyła czas
# wykonania innej funkcji i go zwracała.

# DEKORATOR
# - funkcja, która przyjmuje inną funkcję jako argument, „opakowuje" ją dodatkowym kodem i zwraca nową wersję tej funkcji
# Czyli dekorator pozwala dodać coś do funkcji bez zmieniania jej kodu
    # mierzenie czasu działania funkcji
    # logowanie
    # sprawdzanie uprawnień
    # obsługę błędów

import time
import numpy as np

def measure_time(func):
  def wrapper(*args, **kwargs):
    start_time = time.time() # Zapisuje czas przed uruchomieniem funkcji
    result = func(*args, **kwargs) # Uruchamia właściwą funkcję, którą dekorujemy
    end_time = time.time() # Zapisuje czas po zakończeniu funkcji

    execution_time = end_time - start_time # Oblicza, ile sekund trwało wykonanie funkcji

    print(f"Funkcja {func.__name__} wykonana w {execution_time:.5f} s") # Wypisuje nazwę funkcji i czas działania
    return execution_time # Zwraca czas wykonania
  return wrapper


# 2.Tworzymy listę i tablicę z milionem elementów (liczbami od 0 do 999 999).
size = 1_000_000
py_list = list(range(size))
np_array = np.arange(size)


# 3. Tworzymy funkcję przemnażającą wszystkie elementy listy/tablicy i porównujemy czas wykonania.
print(f"\nMnożenie:")
@measure_time
def standard_multiplication(python_list):
  py_result = [x * 2 for x in py_list]


@measure_time
def numpy_multiplication(numpy_array):
  np_result = np_array * 2

time1 = standard_multiplication(py_list)
time2 = numpy_multiplication(np_array)
print(f"Przyśpieszenie przy mnożeniu {time1/time2:.2f}x")


# 4. Tworzymy funkcję sumującą wszystkie elementy listy/tablicy i porównujemy czas wykonania.
print(f"\nSumowanie:")
@measure_time
def standard_sum(python_list):
  py_sum = sum(py_list)

@measure_time
def numpy_sum(numpy_array):
  np_sum = np.sum(np_array)

time1 = standard_sum(py_list)
time2 = numpy_sum(np_array)
print(f"Przyśpieszenie przy dodawaniu {time1/time2:.2f}x")


# 5. Tworzymy funkcję obliczającą średnią listy/tablicy i porównujemy czas wykonania.
print(f"\nŚrednia:")
@measure_time
def standard_mean(python_list):
  py_mean = sum(py_list) / len(py_list)

@measure_time
def numpy_mean(numpy_array):
  np_mean = np.mean(np_array)

time1 = standard_mean(py_list)
time2 = numpy_mean(np_array)
print(f"Przyśpieszenie przy wyliczaniu średniej {time1/time2:.2f}x")


# 6. Tworzymy funkcję filtrującą listę/tablicę i porównujemy czas wykonania.
print(f"\nFiltrowanie:")
@measure_time
def standard_conditional(python_list):
  py_filtered = [x for x in py_list if x > 500_000]

@measure_time
def numpy_conditional(numpy_array):
  np_filtered = np_array[np_array > 500_000]

time1 = standard_conditional(py_list)
time2 = numpy_conditional(np_array)
print(f"Przyśpieszenie przy filtrowaniu {time1/time2:.2f}x")
```
```
Mnożenie:
Funkcja standard_multiplication wykonana w 0.08685 s
Funkcja standard_multiplication wykonana w 0.08685 s
Funkcja numpy_multiplication wykonana w 0.00465 s
Przyśpieszenie przy mnożeniu 18.68x

Sumowanie:
Funkcja standard_sum wykonana w 0.00711 s
Funkcja numpy_sum wykonana w 0.00092 s
Przyśpieszenie przy dodawaniu 7.70x

Średnia:
Funkcja standard_mean wykonana w 0.00732 s
Funkcja numpy_mean wykonana w 0.00117 s
Przyśpieszenie przy wyliczaniu średniej 6.25x

Filtrowanie:
Funkcja standard_conditional wykonana w 0.04039 s
Funkcja numpy_conditional wykonana w 0.00398 s
Przyśpieszenie przy filtrowaniu 10.16x
```

9. PRAKTYCZNE ZADANIE

In [14]:
```
# Praktyczne wykorzystanie biblioteki numpy - rozwiązywanie układu równań z 3 niewiadomymi.
# https://matematykadlastudenta.pl/strona/474.html

macierz_wspolczynnikow = np.array([ [1, 1, 1],
                                    [2, 2, -2],
                                    [1, 2, 3]])

macierz_rozwiazan = np.array([6, 0, 14])

# Krok 1. Wyznacznik macierzy głównej
def wyznacznik_3x3(matrix):
  det = (
      matrix[0][0] * matrix[1][1] * matrix[2][2]
      + matrix[0][1] * matrix[1][2] * matrix[2][0]
      + matrix[0][2] * matrix[1][0] * matrix[2][1]
      - matrix[0][2] * matrix[1][1] * matrix[2][0]
      - matrix[0][1] * matrix[1][0] * matrix[2][2]
      - matrix[0][0] * matrix[1][2] * matrix[2][1]
  )
  return det

wyznacznik_glowny = wyznacznik_3x3(macierz_wspolczynnikow)
print(f"Wyznacznik macierzy głównej: {wyznacznik_glowny}")

# Krok 2. Macierze X, Y, Z
macierz_x = macierz_wspolczynnikow.copy()
macierz_x[:, 0] = macierz_rozwiazan

macierz_y = macierz_wspolczynnikow.copy()
macierz_y[:, 1] = macierz_rozwiazan

macierz_z = macierz_wspolczynnikow.copy()
macierz_z[:, 2] = macierz_rozwiazan


# Krok 3. Wyznaczniki macierz X, Y, Z
wyznacznik_x = wyznacznik_3x3(macierz_x)
print(f"Wyznacznik macierzy x: {wyznacznik_x}")

wyznacznik_y = wyznacznik_3x3(macierz_y)
print(f"Wyznacznik macierzy y: {wyznacznik_y}")

wyznacznik_z = wyznacznik_3x3(macierz_z)
print(f"Wyznacznik macierzy z: {wyznacznik_z}")


# Krok 4. Obliczenie rozwiązań
if(wyznacznik_glowny != 0):
  x = wyznacznik_x / wyznacznik_glowny
  y = wyznacznik_y / wyznacznik_glowny
  z = wyznacznik_z / wyznacznik_glowny
  print(f"Rozwiązanie układu 3x3: x={x}, y={y}, z={z}")
else:
  print("Układ nieoznaczony")
```
```
Wyznacznik macierzy głównej: 4
Wyznacznik macierzy x: 4
Wyznacznik macierzy y: 8
Wyznacznik macierzy z: 12
Rozwiązanie układu 3x3: x=1.0, y=2.0, z=3.0
```